
Spatio-Temporal databases/data warehouses

Knowledge objectives

1. Explain the main reasons for having spatio-temporal support in DW.
2. Define spatio-temporal DB.
3. Enumerate two main metadata objects to interpret spatial information.
4. Explain two main approaches for representing spatial information.
5. Enumerate three standards/taxonomies for spatial data modeling.
6. Name three basic spatial data types.
7. Distinguish five groups of spatial operators.
8. Enumerate at least five topological operators in spatial DW and how are they used to model spatial dimensions.
9. Give at least one example of distributive, algebraic, and holistic spatial measure.
10. Enumerate three main elements to support modeling of temporal data.
11. Enumerate the two main dimensions of time that temporal databases can model.
12. Distinguish three types of temporal databases wrt. what time dimension they can interpret.
13. Name two main characteristics of DW that make temporal DB suitable for its implementation.
14. Explain the three main components of a temporal DBMS in practice.

Application Objectives

1. Given a temporal aspect of data in a domain at hand, model the DW using temporal DBMS capabilities.
2. Given a geographic information in a domain at hand, complement descriptive information in a DW with spatial component to allow specific spatial analysis over data.

MOTIVATION AND DEFINITION

General motivation

- ❑ Most of human and organizational behavior has some indication of time and space.
 - 80% of data have a spatial or location component
 - Time as an important aspect of real-world concepts
- ❑ Enable historical (time-varying) analysis of data
- ❑ Enable the notion of time-validity
- ❑ Enable spatial relationships in the analysis
 - E.g., airport belongs to a city, city belongs to a province, province to a region, and region to a country.
- ❑ Enable efficiently representing spatial and temporal information
 - Alphanumerics are not enough
- ❑ Optimize data access for spatio-temporal analysis

Definition

- Spatial (and temporal) databases
 - Databases that enable specific constructs (**data types, operations/predicates**) to describe how the real-world phenomena represented in the database are **positioned in a given spatial and temporal framework**.
 - Specific means to process spatial (and temporal) information related to the main data.

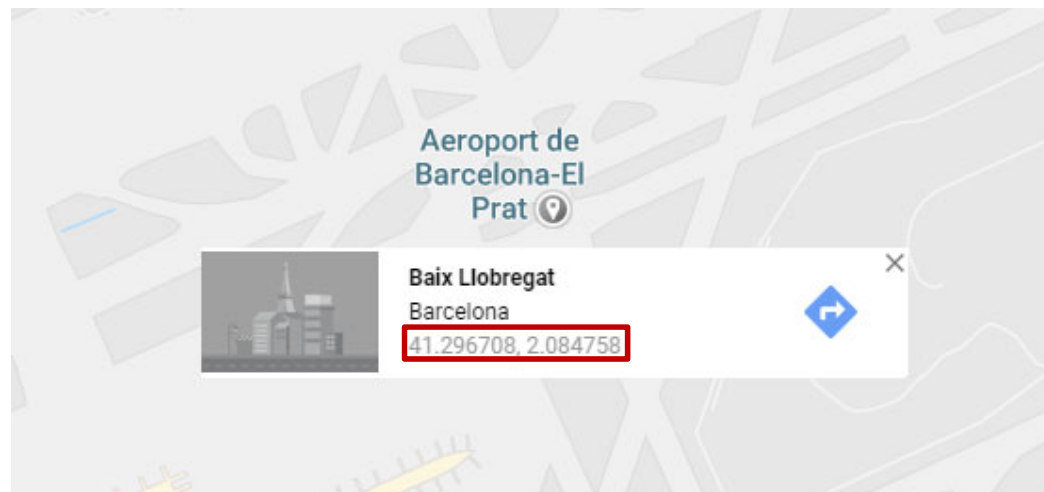
Example

- ❑ Measures that analyze aircraft utilization (FH,FC,delays) can have:
 - Temporal information: time, day, month of completed flights, seasonal flight schedule
 - Spatial information: airport, city, country where the flights operated, trajectory of the flights
- ❑ Analyze historical **trends of delays** or the area with more **density of air traffic** in the last **months/years**, from all the European **airports/cities/countries** where the company operates
- ❑ We need spatio-temporal support to efficiently perform such analysis

SPATIAL DATABASES

Introduction

- ❑ Extending traditional schema to include the specification of a spatial extent
 - E.g., Adding position (coordinates) to the name and address of an airport.
- ❑ Additional metadata to know how to interpret spatial information
 - Reference system
 - Spatial data type
- ❑ Spatial data access
 - Spatial indexing



Applications domains

- ❑ Geographic Information Systems (GIS)
 - Representing geographic objects with different geometries
 - ❑ Points – cities, places of interests, Lines – roads, rivers, Polygons – regions, lakes
- ❑ Multimedia processing
 - Processing image, video, etc
 - Similarity search and patten matching
- ❑ Proximity-based services
 - Nearest point of interest
- ❑ Data Warehousing
 - Geographic hierarchy
 - ❑ City -> Province -> State -> Country -> Continent -> World

Object-based vs. field-based

❑ Object-based approach

- Identifiable objects in space with described shapes
 - ❑ Road as a line
 - ❑ State as a surface/polygon

❑ Field-based approach

- Phenomena that vary on space
- Each point of a shape has a value
 - ❑ Temperatures across different regions in a country,
 - ❑ Altitudes in a country.

SPATIAL DATA TYPES

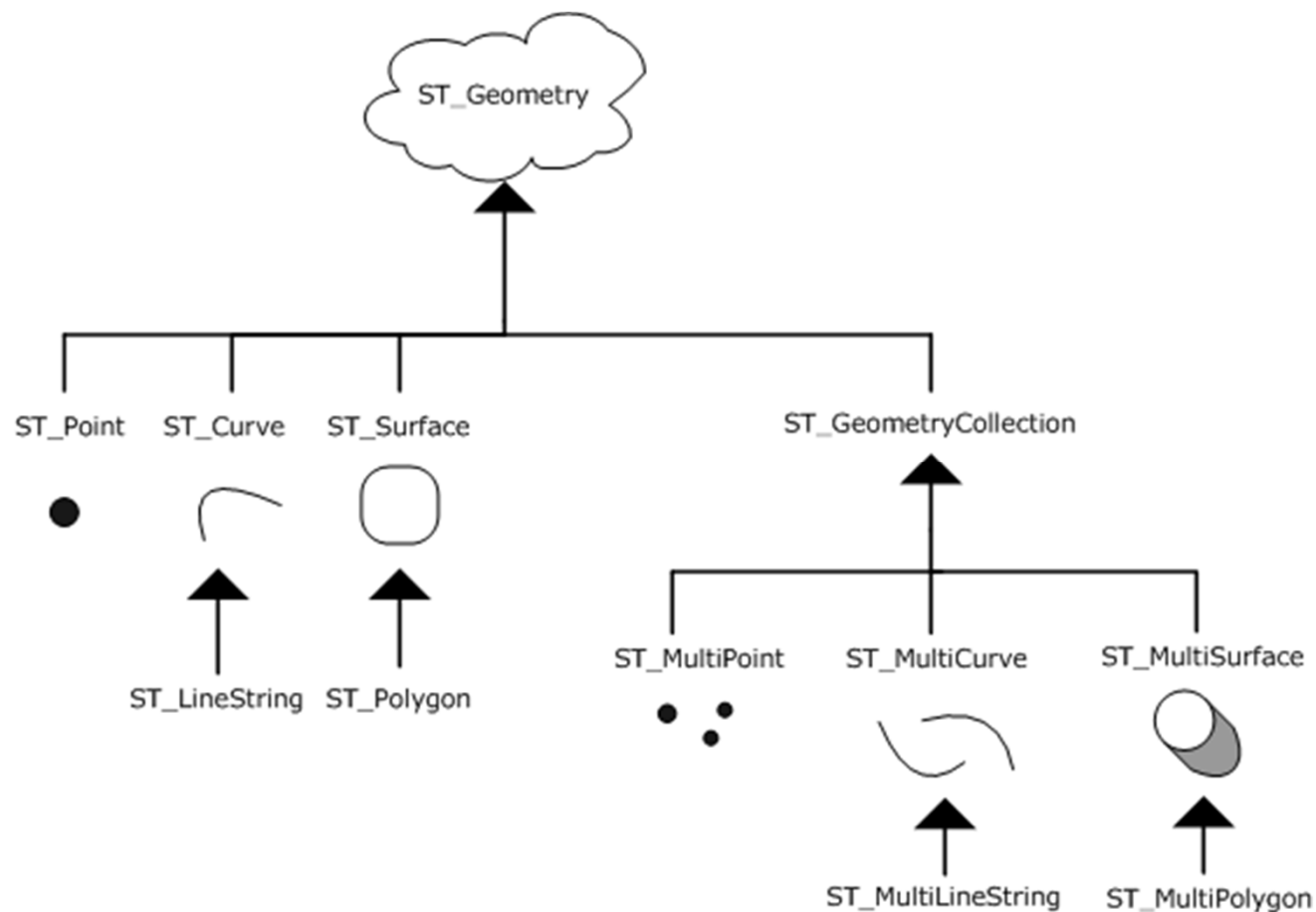
Spatial data types

- ❑ Real-world entity that carries some spatial characteristics
- ❑ Two components
 - Descriptive – traditional data types (name, population, capital, etc.)
 - Spatial – extent of object in the space (framework) of interest
- ❑ ISO TC 211 standard
- ❑ OGC standard
 - implemented in majority of spatial DBs
- ❑ MADS spatio-temporal model
 - Conceptual model!
 - Extended for spatial DW

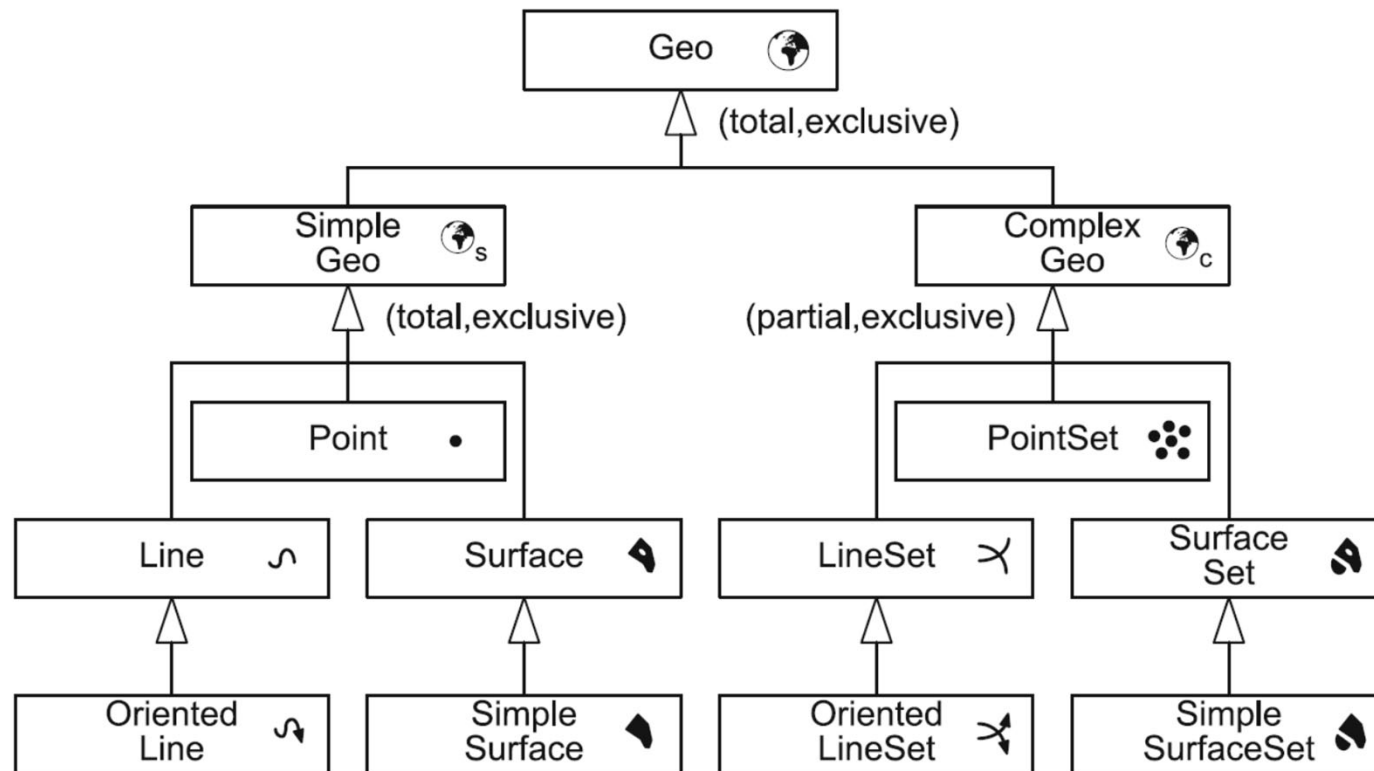
ST_Geometry

- ❑ Efficient use of database resources
- ❑ Compatible with other DB features
 - E.g., Replication and partitioning
- ❑ Enables rapid access to spatial data
- ❑ Common and subtype specific properties and functions
- ❑ Two main subtypes
 - Base geometry
 - ❑ ST_Point, ST_LineString, ST_Polygon
 - Homogeneous collection
 - ❑ ST_MultiPoint, ST_MultiLineString, ST_MultiPolygon

Spatial data types – OGC standard

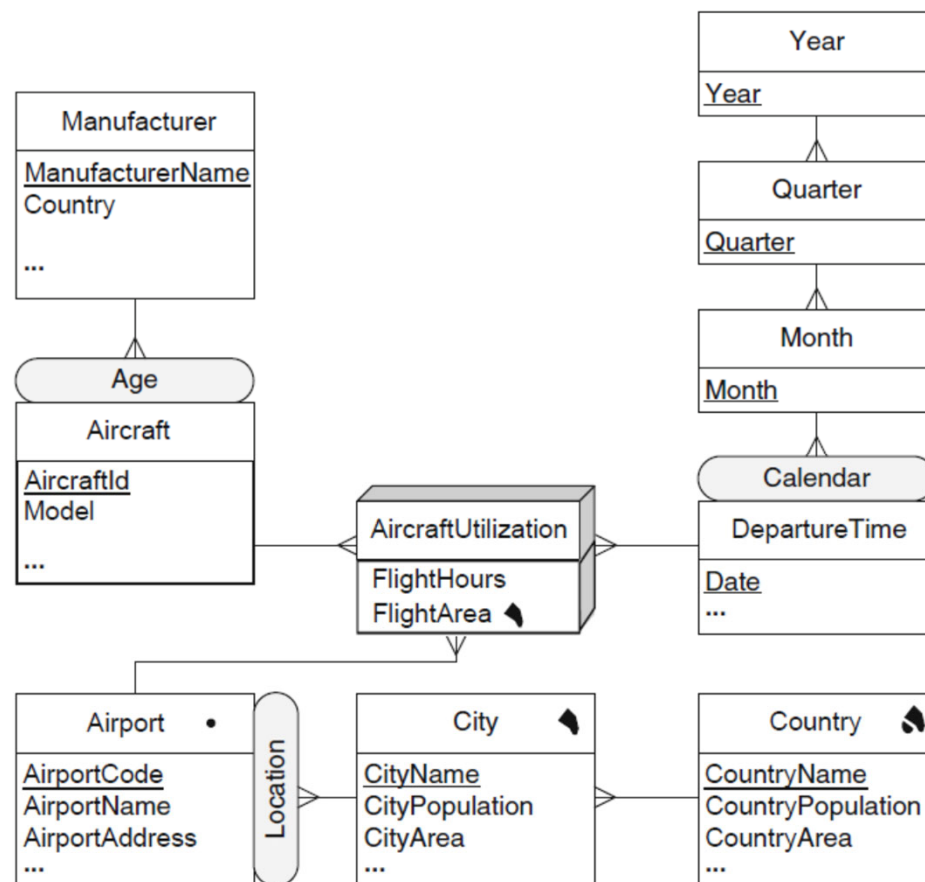


Spatial data types - MODS data model



Alejandro A. Vaisman, Esteban Zimányi: Data Warehouse Systems - Design and Implementation. Data-Centric Systems and Applications, Springer 2014, ISBN 978-3-642-54654-9, pp. 1-588

MODS data model - Example

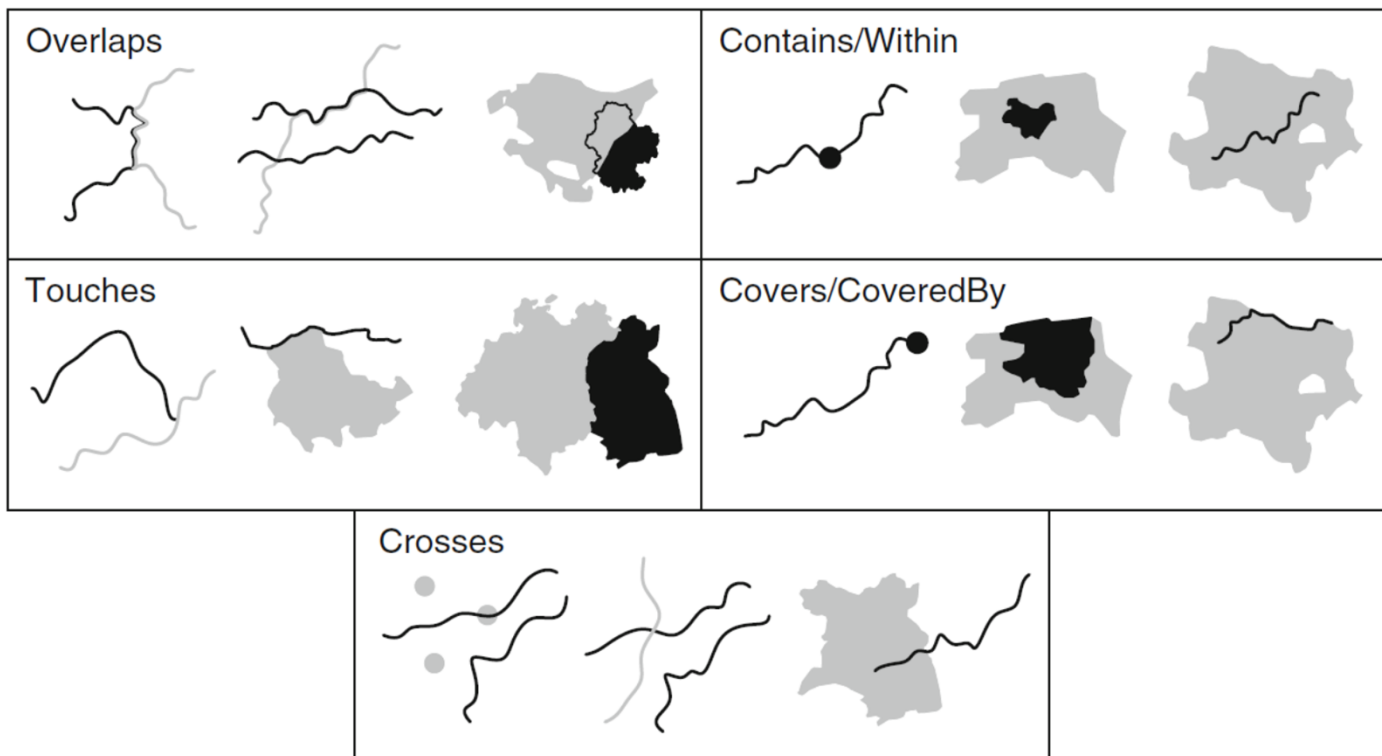


SPATIAL OPERATIONS

Spatial operations

- ❑ Numeric
 - NoComponents, Length, Area, Perimeter, Distance, Direction
- ❑ Predicates
 - IsEmpty, OnBorder, InInterior
- ❑ Unary operations
 - Boundary, Buffer, Centroid, ConvexHull
- ❑ Binary operations
 - Intersection, Union, Difference, SymDifference, **Spatial join**
- ❑ Topological operations
 - Intersects, Disjoint, **Equals**, Overlaps, **Contains**, **Within**, Touches, **Covers**, **CoveredBy**, Crosses

Topological operations - examples

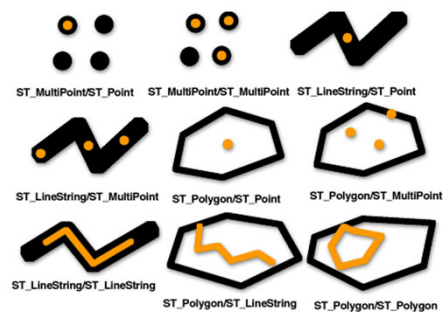


Alejandro A. Vaisman, Esteban Zimányi: Data Warehouse Systems - Design and Implementation. Data-Centric Systems and Applications, Springer 2014, ISBN 978-3-642-54654-9, pp. 1-588

Topological operations

- ❑ Comparing two geometry objects by means of:
 - The **exterior** of the geometries, which is all of the space not occupied by a geometry
 - The **interior** of the geometries, which is the space occupied by a geometry
 - The **boundary** of the geometries, which is the interface between a geometry's interior and exterior
- ❑ Expresses spatial/topological relationships among all types of geometry as **pair-wise intersections** of their **interior**, **boundary**, and **exterior**
 - Considering the **dimension** of the resulting intersections
 - ❑ -1 – none; 0 – point; 1 – line; 2 – polygon
 - Comparing it to a **pattern matrix** of values
 - ❑ T – Intersection of any dimension must exist (0,1,2)
 - ❑ F – Intersection must not exist (-1)
 - ❑ * – It does not matter if an intersection exists or not (-1,0,1,2)
 - ❑ 0, 1, 2 – an intersection of the given dimension (0,1,2) must exist

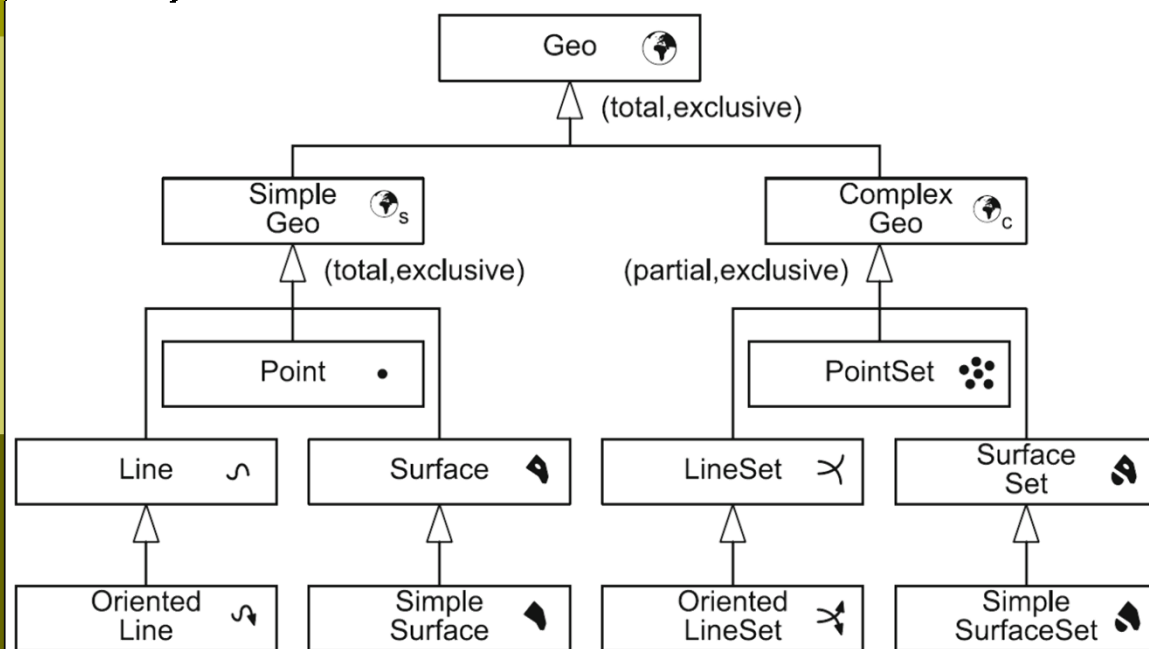
Example: ST_Contains



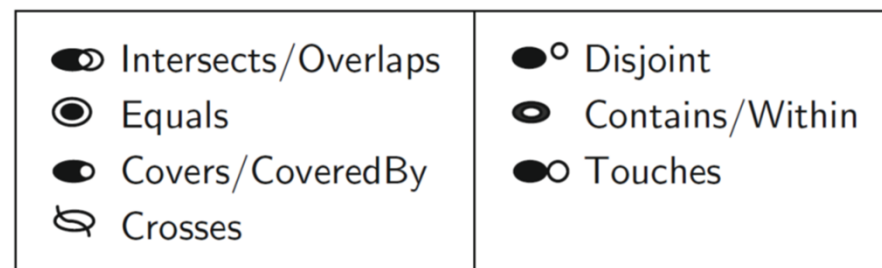
		b		
		Interior	Boundary	Exterior
a	Interior	T	*	*
	Boundary	*	*	*
	Exterior	F	F	*

Conceptual modeling of spatial data warehouses

MODS ST model: Icons for spatial data types and spatial operations

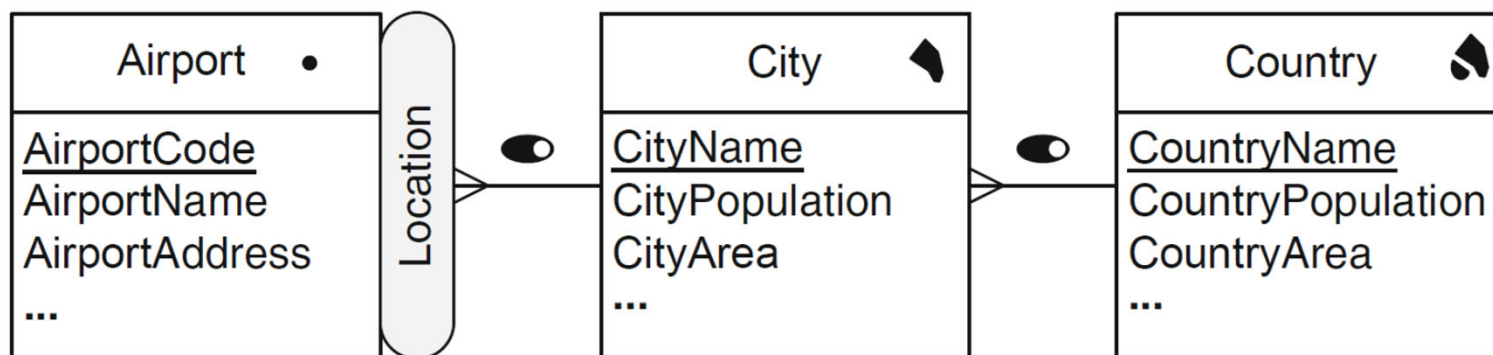


Spatial data types icons



Spatial topological operations icons

Spatial hierarchies



Examples inspired by: Alejandro A. Vaisman, Esteban Zimányi: Data Warehouse Systems - Design and Implementation. Data-Centric Systems and Applications, Springer 2014, ISBN 978-3-642-54654-9, pp. 1-588

Spatial measures

□ Dimensions can be spatial or not

□ Spatial aggregations

■ Distributive:

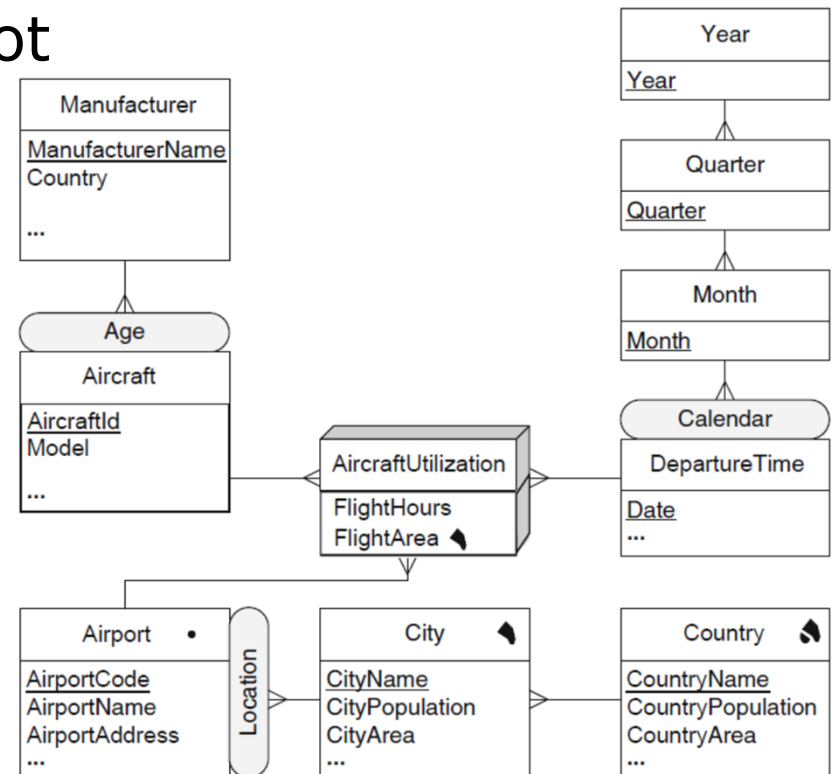
□ convex hull, spatial union, spatial intersection

■ Algebraic:

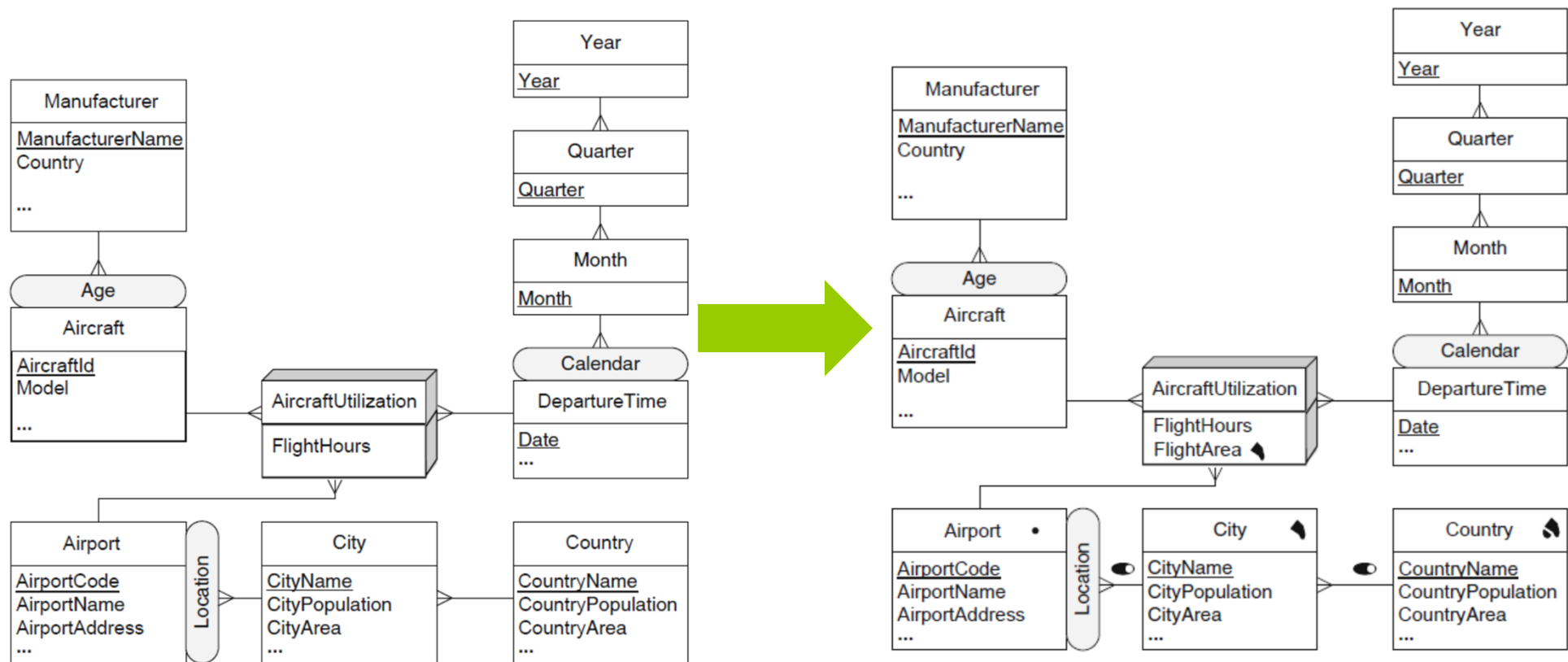
□ center of n points

■ Holistic:

□ equipartition



Examples inspired by: Alejandro A. Vaisman, Esteban Zimányi: Data Warehouse Systems - Design and Implementation. Data-Centric Systems and Applications, Springer 2014, ISBN 978-3-642-54654-9, pp. 1-588

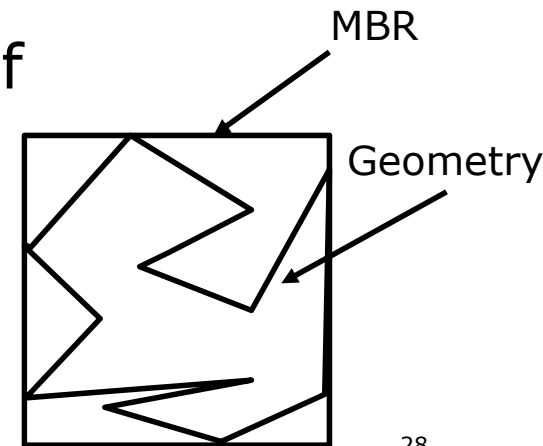


Examples inspired by: Alejandro A. Vaisman, Esteban Zimányi: Data Warehouse Systems - Design and Implementation. Data-Centric Systems and Applications, Springer 2014, ISBN 978-3-642-54654-9, pp. 1-588

SPATIAL INDEXING TECHNIQUES

Background

- ❑ Efficiently support spatial operations and predicates
 - e.g., overlaps, nearest neighbor, spatial joins
- ❑ Specialized access methods (indexes) proposed in order to optimize complex spatial queries
- ❑ Processing complex geometries (e.g., multi polygons with multitude of points) is CPU-intensive
- ❑ Geometry objects approximated by means of
 - Minimum Boundary Rectangle (MBR)
- ❑ MBRs used inside the spatial indexes instead of original geometries
 - Actual geometries stored separately



Spatial index classification

- ❑ Based on the space they cover
 - Space-driven structures: Partitions the complete space under consideration while spatial objects are mapped to the space partitions
 - Data-driven structures: Partitions only the part of the space containing the spatial objects
- ❑ Based on the object indexed
 - Point access methods (multidimensional points)
 - Spatial access methods (multidimensional regions)

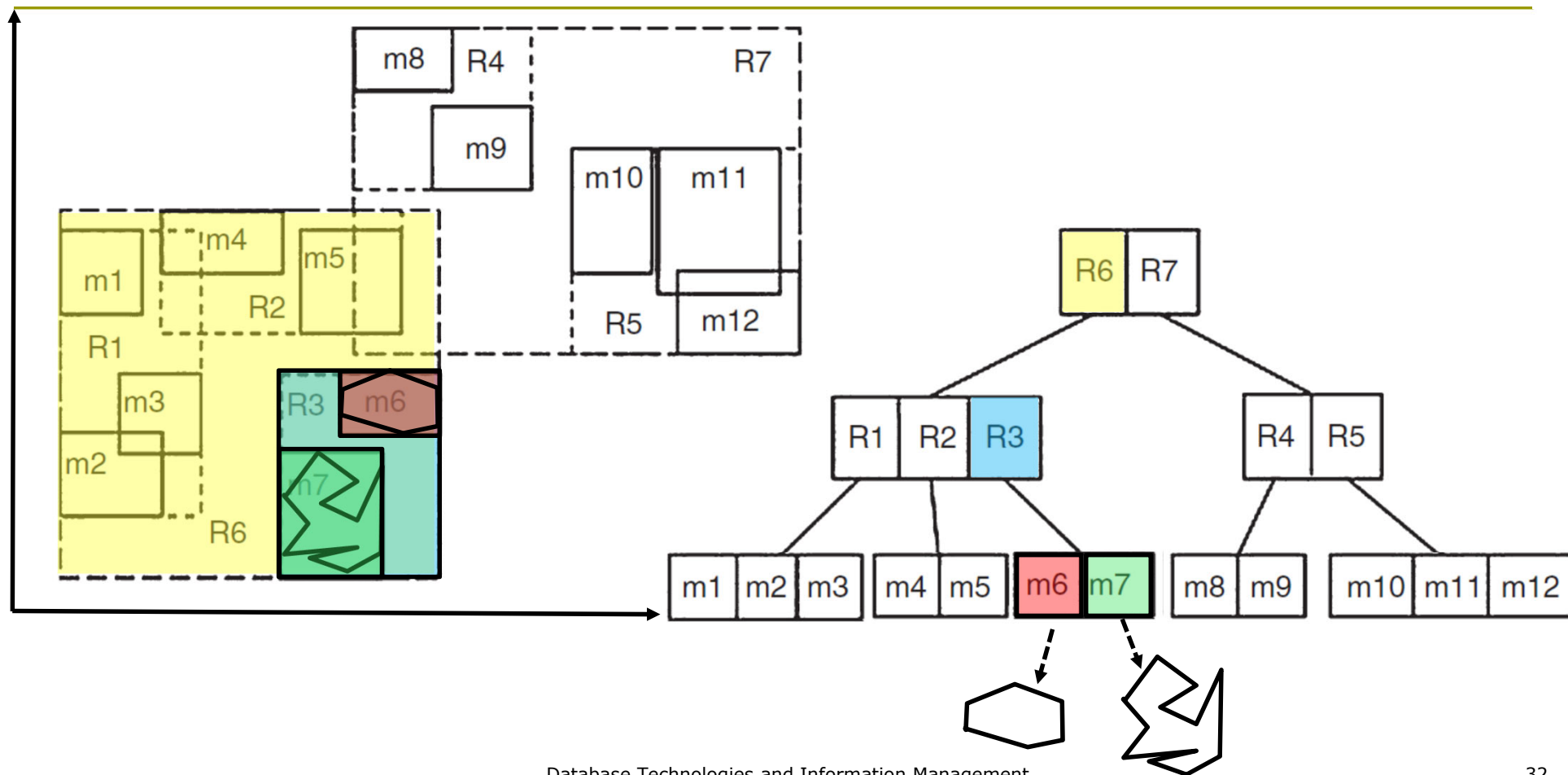
Spatial indexes - examples

- R-tree
 - Data-driven / Spatial access method
- Quadtree
 - Space-driven / Spatial access method
- LSD-tree
 - Space-driven / Point access method

R-tree

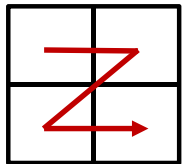
- Height-balanced tree (based on B⁺-tree) that consists of:
 - **Leaf nodes** consist of collection of pairs (reference to the geometry object in a DB, MBR of that geometry object)
 - **Intermediate nodes** consist collection of pairs (reference to the child node in an R-tree, MBR enclosing all MBRs of the child node).
- Filter/join queries
 - Following the MBRs from the root of the R-tree (from coarser to finer level) until the leaf level is reached
- Inserting new geometry
 - Again, from the root of the R-tree, follow the child node that needs **minimum enlargements** to fit geometry's MBR, until the leaf is reached
 - If leaf overfilled, node is split and the entries are redistributed
 - Splits also follow the minimum enlargements criteria

R-tree - example

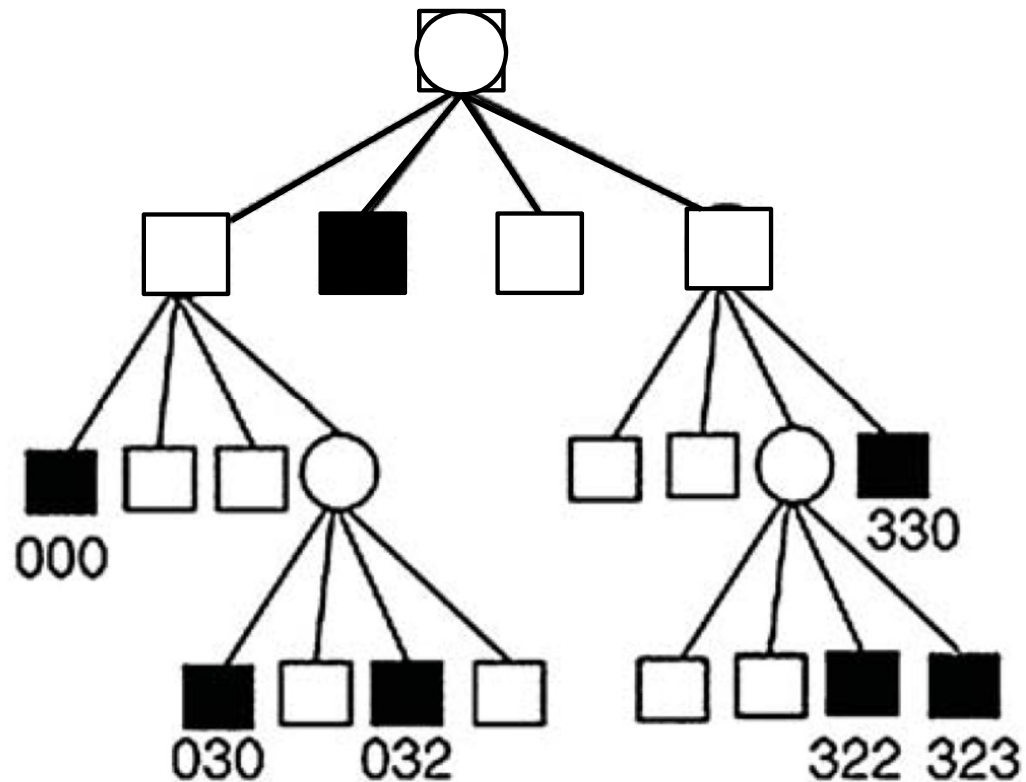
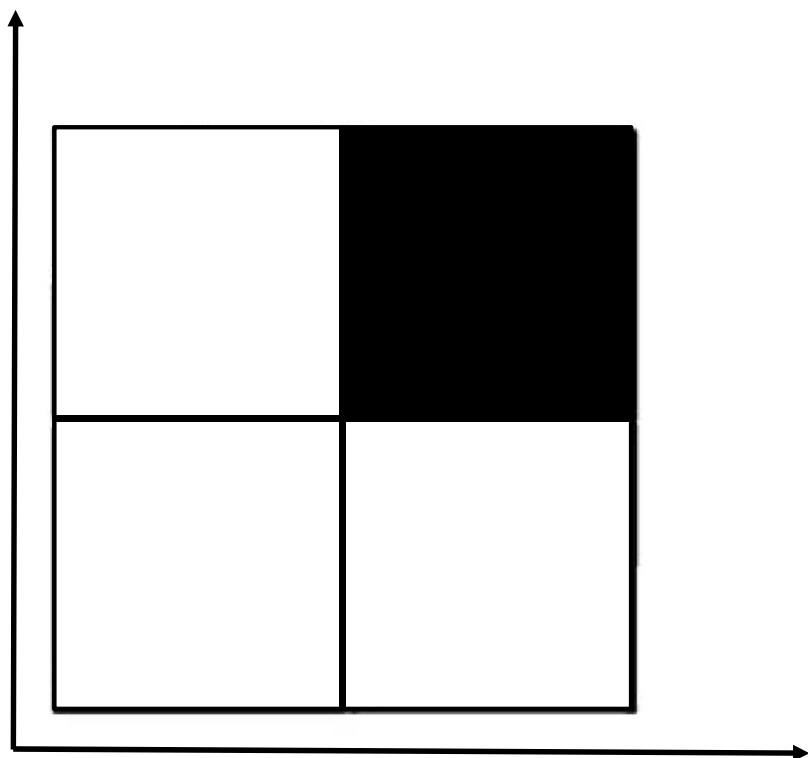


Quadtree

- ❑ Typically used to for representing binary (black & white) images
 - $2^n \times 2^n$ matrix (1 – black, 0 – white)
- ❑ Uses degree four tree of max height **n**
 - The root corresponds to the whole image (space)
 - Each node corresponds to the square array of matrix cells (pixels)
 - ❑ If all the pixels have the same color, the node is **leaf** , with the color of its pixels
 - ❑ Otherwise, the node is intermediate , with 4 children nodes, one for each of its subcomponent cells (from upper left to lower right)
- ❑ Variations for multicolor images



Quadtree - example



TEMPORAL DATABASES

Introduction

- Data values can vary over time
- Inherent need to analyze data over time
 - Following trends
- , or at a specific moment in time
 - Taking snapshots
- Need specific support to model temporal data
 - Expressing various dimensions of time
 - Transaction, Valid, User-defined time
 - Querying “through” time(s)

Application domains

- ❑ Financial
 - accounting & banking, stock market analysis, audit analysis
- ❑ Record-keeping
 - organization personnel, medical history, inventory management, legal records
- ❑ Scheduling
 - airlines, car rentals, hotel reservation, project management
- ❑ Science
 - weather monitoring, chemical process monitoring
- ❑ Data Warehousing / Decision Support Systems
 - historical information for trend analysis/forecasting

Theoretical foundations of temporal DBs

- ❑ Incorporating temporal information in the data management
- ❑ Can store and interpret up to two dimensions of time
 - TRANSACTION (implicitly logged) time
 - VALID (explicitly stated) time
- ❑ User-defined time
 - Semantics specific/known by user
 - Not interpreted by DBMS
- ❑ Types of Temporal DBs
 - Uni-temporal (VALID or TRANSACTION)
 - ❑ **Historical** (VALID) or **Rollback** (TRANSACTION)
 - Bi-temporal (VALID and TRANSACTION)

Transaction time support

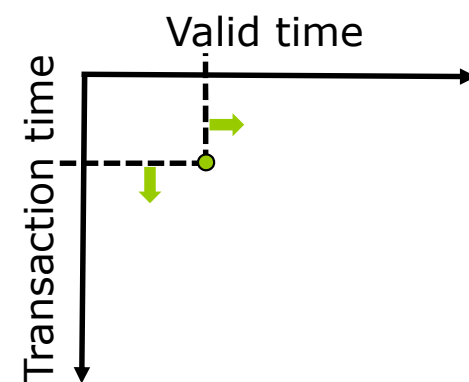
- ❑ Time evolution in terms of database transactions occurred
- ❑ Models database reality
 - insertions, updates, deletion
- ❑ Database records extended with a time interval
- ❑ On *insert*
 - Adds [t1, now)
- ❑ On *update*
 - Update the current record: [t1, now) -> [t1, t2]
 - Replicate the record adding: [t2, now)
- ❑ On *delete* (logical)
 - Update the current record: [t2, now) -> [t2, t3]

Valid time support

- ❑ Models real world phenomena
 - past, present, future
- ❑ Valid-time column(s)
 - start/end timestamps vs period data type
- ❑ Explicitly stated dates/intervals
- ❑ Operation over the interval data
 - Deletion (any time) – physical
 - Insertion (any time)
 - Updates (any time)

Bi-temporal support

- Both valid time columns and transaction time columns
- Valid and transaction time dimensions separately considered when modifying (updating/deleting) data
- Example, on row delete:
 - End bound of transaction time updated to current time
 - Valid time period remains unchanged
 - Row becomes disabled for further DML operations
- Database snapshots must be made from both valid and transaction time point of view.



Temporal data in DW

- ❑ By definition DW is a **time-variant** and **non-volatile** collection of data.
 - Historical data.
 - Trends analysis though time
- ❑ Time dimension with multiple temporal levels
 - minute, hour, day, month, quarter, year
- ❑ Also, time-varying dimension data
 - Slowly changing dimensions
- ❑ Temporal DB can enable efficiently storing and analyzing data with regard to time-varying information
 - Temporal Data warehousing

Temporal DBMS

- Temporal Data Models
 - Adding temporal attributes to DB relations
- Temporal Query Languages
 - Temporal SQL extensions
 - TQUEL, SQL3
- Temporal Query Processing
 - Temporal joins: time-equijoin, event-join, TE-outer join, contain-join, contain-semi join, temporal natural join and intersect-join
 - Range queries with R-tree or B-tree indexes

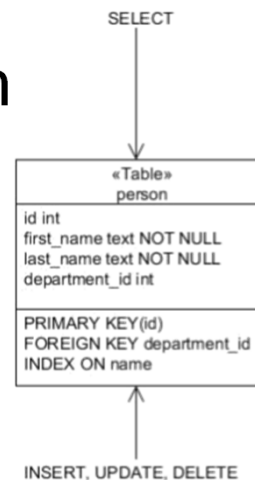
Temporal DBMS: Oracle

- ❑ From v12c, Oracle is completely “time-aware”
- ❑ Bi-temporal support
 - Transaction time – flashback
 - Valid time – temporal validity
- ❑ Flashback
 - Query trends
 - History keeping
 - Navigating in time
 - Recovery
 - Audit trails
- ❑ Temporal validity
 - valid_time markers
 - Two date-time columns model the temporal validity
 - Possible multiple per table

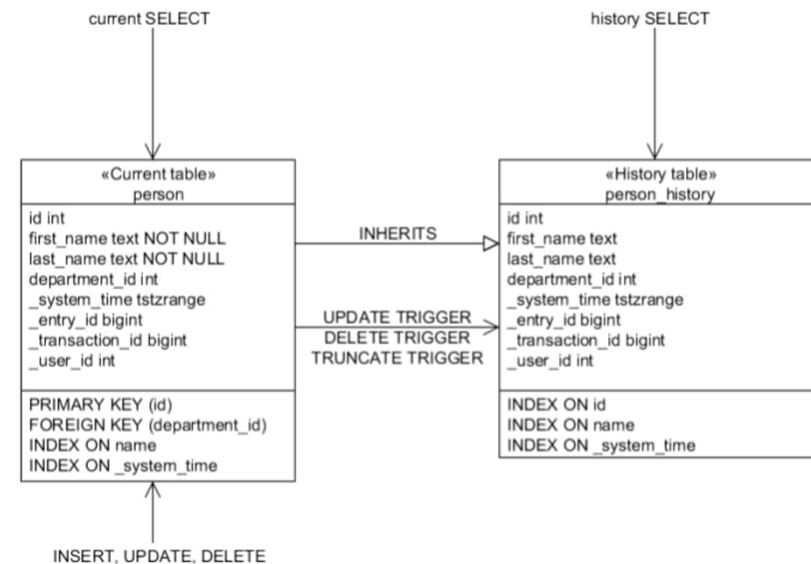
Temporal DBMS: PostgreSQL

- ❑ Support through System Period Temporal Tables
 - ❑ Maintain historical versions of data
 - Current table
 - Historical table
- ❑ Modification operation
 - Create new historical records
- ❑ Range data type
 - Model temporal validity of records
 - $[t1, \infty)$
 - $[t1, t2]$

Regular table



System period temporal table



Temporal DBMS - example

□ SQL3 support for querying with valid-time

```
CREATE TABLE Employee (
  Name VARCHAR(30),
  Manager VARCHAR(30) REFERENCES Employee (Name),
  Dept VARCHAR(20));
```



```
SELECT Name
FROM Employee
EXCEPT
SELECT Manager
FROM Employee
```

```
SELECT Dept, COUNT(*)
FROM Employee
GROUP BY Dept
```

```
CREATE TABLE Employee (
  Name VARCHAR(30),
  Manager VARCHAR(30) VALIDTIME REFERENCES Employee (Name),
  Dept VARCHAR(20)) AS VALIDTIME PERIOD(DATE);
```



```
VALIDTIME SELECT Name
FROM Employee
EXCEPT
SELECT Manager
FROM Employee
```

```
VALIDTIME SELECT Dept, COUNT(*)
FROM Employee
GROUP BY Dept
```

Temporal DBMS - example

▣ SQL3 support for updates with valid-time

```
UPDATE Employee  
SET Manager = 'Bob'  
WHERE Dept = 'Tools'
```

```
VALIDTIME PERIOD '[1994-01-01 - 1994-12-31]'  
UPDATE Employee  
SET Manager = 'Bob'  
WHERE Dept = 'Tools'
```

Temporal DBMS – temporal join example 1

EmployeeSalary

Name	Salary	VT _{start}	VT _{end}
Jordi	1000	2020-7-1	2020-7-10
Jordi	2000	2020-7-11	2020-7-20
Jordi	3000	2020-7-21	2020-7-30

EmployeeDepartment

Name	Department	VT _{start}	VT _{end}
Jordi	ESSI	2020-7-1	2020-7-15
Jordi	AC	2020-7-16	2020-7-20

```
SELECT es.Name, es.Salary, ed.Department
FROM EmployeeSalary es, EmployeeDepartment ed
WHERE es.Name = ed.Name
```

Name	Salary	Department	VT _{start}	VT _{end}
Jordi	1000	ESSI	2020-7-1	2020-7-10
Jordi	2000	ESSI	2020-7-11	2020-7-15
Jordi	2000	AC	2020-7-16	2020-7-20
Jordi	3000	AC	2020-7-21	2020-7-30

Temporal DBMS – temporal join example 2

- ❑ Without temporal DB support
 - Start / Stop columns (instead of VT)
- ❑ Answering temporal-join requires more complex queries

```

SELECT es1.Name, es1.Salary, ed1.Department,
       es1.Start, es1.Stop
FROM EmployeeSalary es1, EmployeeDepartment ed1
WHERE es1.Name = ed1.Name
      AND ed1.Start <= es1.Start
      AND es1.Stop < ed1.Stop
UNION
SELECT es2.Name, es2.Salary, ed2.Department
FROM EmployeeSalary es2, EmployeeDepartment ed2
WHERE es2.Name = ed2.Name
      AND es2.Start >= ed2.Start
      AND ed2.Stop < es2.Stop
      AND es2.Start < ed2.Stop
UNION

```

...

```

...
SELECT es3.Name, es3.Salary, ed3.Department
FROM EmployeeSalary es3, EmployeeDepartment ed3
WHERE es3.Name = ed3.Name
      AND ed3.Start > es3.Start
      AND es3.Stop < ed3.Stop
      AND ed3.Start < es3.Stop
UNION
SELECT es4.Name, es4.Salary, ed4.Department
FROM EmployeeSalary es4, EmployeeDepartment ed4
WHERE es4.Name = ed4.Name
      AND ed4.Start > es4.Start
      AND ed4.Stop < es4.Stop;

```

CLOSING

Summary I

- ❑ Human and organizational behavior often has indication of time and space
- ❑ Spatial and temporal specific components
 - Data types, Operators, Predicates
- ❑ Spatial DB/DW
 - **Object-based** approach to model spatial information (coordinates, polygons), **field-based** approach to model varying phenomena over the space (temperature, altitude)
 - Spatial data modeling – point, line/curve, surface/polygon
 - Spatial operators – topological, unary, binary, predicates
 - Spatial DW modeling – spatial hierarchies and spatial measures
- ❑ Spatial indexing
 - Point vs. Spatial based and Data-driven vs. Space-driven
 - R-tree and Quadtree

Summary II

□ Temporal DB/DW

- Transaction vs. valid dimension of time.
- Uni-temporal (historic or rollback) vs. bi-temporal DB
- Temporal DBMS (data model, query language, query processing)
 - Oracle
 - PostgreSQL
- DW as time-variant and non-volatile data collection
 - Commercially not exploiting temporal DBMS capabilities.

Bibliography

- ❑ Alejandro A. Vaisman, Esteban Zimányi: Data Warehouse Systems - Design and Implementation. Data-Centric Systems and Applications, Springer 2014, ISBN 978-3-642-54654-9, pp. 1-588
- ❑ C. Parent, S. Spaccapietra, E. Zimanyi, *Conceptual Modeling for Traditional and Spatio-Temporal Applications: The MADS Approach* (Springer, 2006)
- ❑ Vaisman A.A., Zimányi E. (2018) Temporal Datawarehousing. In: Encyclopedia of Database Systems. Springer.
- ❑ Manolopoulos Y., Theodoridis Y., Tsotras V.J. (2018) Spatial Indexing Techniques. In: Encyclopedia of Database Systems. Springer.
- ❑ S. Shekhar and S. Chawla. Spatial Databases: A Tour. Prentice Hall, Upper Saddle River, NJ, 2003.
- ❑ OGC standard: <https://www.opengeospatial.org/>
- ❑ ISO standard: <https://www.iso.org/committee/54904.html>